Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to add Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to Implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service From jQuery
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- ASP.NET Web API Routing
- Routing Variations in WEB API
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- Route Prefix in Web API
- Web API Attribute Routing Route Constraints
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Role-Based Basic Authentication in Web API
- Web API Service with Basic Authentication
- HTTP Message Handlers in WEB API
- Basic Authentication Using Message Handler in Web API
- HTTP Client Message Handler in Web API
- Token Based Authentication in ASP.NET Web API
- Client Validation Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- HMAC Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Enable HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

Route Names and Route Orders in Attribute Routing

Back to: ASP.NET Web API Tutorials For Beginners and Professionals

Route Names and Route Orders in Attribute Routing

In this article, I will discuss the **Route Names and Route Orders in Attribute Routing** with examples. We are going to work with the same example that we worked in our previous articles. So if you have not read those articles then I strongly recommend you to read the following articles before proceeding to this article.

Attribute Routing in Web API

Optional URI Parameters and Default values in Attribute Routing

Attribute Routing Route Prefix in WEB API

Route Constraints in Attribute Routing

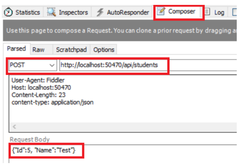
Route Names

In ASP.NET Web API, each and every route has a name. The Route names are useful for generating links so that you can include a link in an HTTP response.

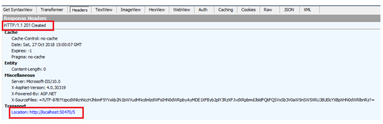
To specify the route name, we need to set the Name property on the attribute. Let us see an example to understand how to set the route name, and also how to use the route name when generating a link.

```
namespace AttributeRoutingInWEBAPI.Controllers
{
    public class StudentsController : ApiController
    {
        static List<Student> students = new List<Student>()
        {
            new Student() { Id = 1, Name = "Pranaya" },
            new Student() { Id = 2, Name = "Priyanka" },
            new Student() { Id = 3, Name = "Anurag" },
            new Student() { Id = 4, Name = "Sambit" }
        };

        [HttpGet]
        [Route("{studentID:nonzero}", Name = "GetStudentById")]
        public Student GetStudentDetails(int studentID)
        {
            Student studentDetails = students.FirstOrDefault(s => s.Id == studentID);
            return studentDetails;
        }

        [Route("api/students")]
        public HttpResponseMessage Post(Student student)
        {
            students.Add(student);
            var response = Request.CreateResponse(HttpStatusCode.Created);
            ----
            // Generate a link for the new student and set the Location header in the
response.
            string uri = Url.Link("GetStudentById", new { studentID = student.Id });
            response.Headers.Location = new Uri(uri);

            return response;
        }
    }
}
```

Let's test this using Fiddler.



Then click on the execute button. It will give us the below result.



Route Order

When the WEB API Framework tries to match a URI with a route, it evaluates the routes in a particular order. To specify the order, set the **Order** property on the route attribute. Lower values are evaluated first. The default order value is zero.

Here is how the total ordering is determined:

1. Compare the **Order** property of the route attribute.
2. Look at each URI segment in the route template. For each segment, order as follows:
3. Literal segments.
4. Route parameters with constraints.
5. Route parameters without constraints.
6. Wildcard parameter segments with constraints.
7. Wildcard parameter segments without constraints.
8. In the case of a tie, routes are ordered by a case-insensitive ordinal string comparison (OrdinalIgnoreCase) of the route template.

Here is an example. Suppose you define the following controller:

```
namespace AttributeRoutingInWEBAPI.Controllers
{
    [RoutePrefix("orders")]
    public class OrdersController : ApiController
    {
        [Route("{id:int}")] // constrained parameter
        public HttpResponseMessage Get(int id)...

        [Route("details")]  // literal
        public HttpResponseMessage GetDetails()...

        [Route("pending", Order = 1)]
        public HttpResponseMessage GetPending()...

        [Route("{customerName}")]  // unconstrained parameter
        public HttpResponseMessage GetByCustomer(string customerName)...

        [Route("{*date:datetime}")]  // wildcard
        public HttpResponseMessage Get(DateTime date)...
    }
}
```

These routes are ordered as follows.

1. orders/details
2. orders/{id}
3. orders/{customerName}
4. orders/{*date}
5. orders/pending

Notice that "details" is a literal segment and appears before ":{id}", but "pending" appears last because the **Order** property is 1. (This example assumes there are no customers named "details" or "pending". In general, try to avoid ambiguous routes. In this example, a better route template for GetByCustomer is "customers/{customerName}")

In this article, I try to explain the **Route Names and Route Orders in Attribute Routing** with examples. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.

Previous Lesson
Web API Attribute Routing Route Constraints

Next Lesson
Authentication and Authorization in ASP.NET Web API

DOT NET TUTORIALS
Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

2 thoughts on "Route Names and Route Orders in Attribute Routing"

RAMSHARAN CHOUDHARY
OCTOBER 9, 2019 AT 12:37 AM

Really Good Work… Appreciate it…

Reply

MANU
APRIL 8, 2021 AT 9:42 PM

Very good explanation . I really fall in love with your blog

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name* | Email* | Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel